RELATÓRIO TÉCNICO: ORIENTAÇÃO A OBJETOS EM DART

Aluno: Willian Raphael

1. INTRODUÇÃO À PROGRAMAÇÃO ORIENTADA A OBJETOS (POO)

A Programação Orientada a Objetos (POO) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados na forma de campos (atributos ou propriedades) e código na forma de procedimentos (métodos). O objetivo principal da POO é aproximar a estrutura do código do mundo real, promovendo reuso, modularidade, flexibilidade e facilidade de manutenção.

2. CONCEITOS FUNDAMENTAIS

2.1. Classe

A classe é o **molde ou gabarito** (blueprint) a partir do qual os objetos são criados. Ela define a estrutura de dados (atributos) e os comportamentos (métodos) que os objetos daquele tipo possuirão.

2.2. Objeto

O objeto é uma **instância concreta** de uma classe. Quando uma classe é instanciada, memória é alocada para armazenar seus valores específicos de atributos.

2.3. Atributos e Métodos

- **Atributos:** Representam o estado ou as características do objeto (ex.: `numeroConta`, `saldo`, `titular`).
- **Métodos:** Representam as ações ou comportamentos que o objeto pode executar (ex.: `depositar()`, `sacar()`, `transferir()`).

2.4. Construtores

Métodos especiais responsáveis por inicializar os atributos de um objeto no momento de sua criação. Em Dart, existem construtores padrão, nomeados (Named Constructors), constantes e construtores de fábrica (Factory Constructors).

3. OS QUATRO PILARES DA POO

3.1. Abstração

Consiste em isolar os aspectos essenciais de uma entidade do mundo real, ignorando detalhes irrelevantes para o contexto da aplicação. Em Dart, a abstração é frequentemente representada por **classes abstratas** (abstract class), que não podem ser instanciadas diretamente e podem conter métodos abstratos (sem assinatura/corpo).

3.2. Encapsulamento

É o mecanismo de proteção dos dados internos de um objeto contra acessos diretos e indevidos do exterior.

- **Em Dart:** O encapsulamento em nível de biblioteca é feito prefixando identificadores com um caractere sublinhado (_).
- **Getters e Setters:** Permitem leitura e escrita controladas sobre atributos privados, viabilizando validações e regras de negócio.

3.3. Herança

Permite que uma classe (classe filha/subclasse) herde atributos e métodos de outra classe (classe pai/superclasse), promovendo o reuso de código e a criação de hierarquias de tipos (extends).

3.4. Polimorfismo

Significa "muitas formas". Permite que objetos de diferentes classes derivadas sejam tratados como objetos da superclasse, mas executem comportamentos específicos ao sobrescreverem métodos (@override).

4. RECURSOS ESPECÍFICOS E AVANÇADOS DO DART

- **Interfaces Implicitas (implements):** Em Dart, toda classe define implicitamente uma interface contendo todos os seus membros públicos. Outra classe pode implementar essa interface obrigando-se a fornecer uma implementação para cada membro.
- **Mixins (mixin e with):** Permitem reutilizar o código de uma classe em múltiplas hierarquias sem a necessidade de herança múltipla.
- **Enums com propriedades:** Permitem representar tipos enumerados com atributos e métodos associados.
- **Null Safety (Sound Null Safety):** O sistema de tipos do Dart garante que variáveis não possam conter null, a menos que sejam explicitamente declaradas como anuláveis (?).

CÓDIGO FONTE DA APLICAÇÃO BANCÁRIA EM DART

Abaixo está o código-fonte completo em arquivo único, pronto para ser executado (por exemplo, no DartPad ou via terminal com dart run main.dart).

```
// ==========================================
// 1. CLASSE MÃE / SUPERCLASSE (ABSTRAÇÃO)
// ==========================================
class ContaBancaria {
  // Atributos privados (Encapsulamento)
  String _titular;
  double _saldo;

  // Construtor
  ContaBancaria(this._titular, this._saldo);

  // Getters para leitura segura dos dados
  String get titular => _titular;
  double get saldo => _saldo;

  // Métodos de negócio
  void depositar(double valor) {
    if (valor > 0) {
      _saldo += valor;
      print("Depósito de R\$ ${valor.toStringAsFixed(2)} realizado com sucesso!");
    } else {
      print("Valor de depósito inválido.");
```

```
  }
}

 void sacar(double valor) {
  if (valor > 0 && valor <= _saldo) {
    _saldo -= valor;
    print("Saque de R\$ ${valor.toStringAsFixed(2)} realizado com sucesso!");
  } else {
    print("Saldo insuficiente ou valor inválido para saque.");
  }
}

 void exibirExtrato() {
  print("--- EXTRATO ---");
  print("Titular: $_titular");
  print("Saldo atual: R\$ ${_saldo.toStringAsFixed(2)}");
}
}

// ========================================
// 2. CLASSE FILHA (HERANÇA E POLIMORFISMO)
// ========================================
class ContaPoupanca extends ContaBancaria {
 int _diaAniversario;

 ContaPoupanca(String titular, double saldo, this._diaAniversario)
    : super(titular, saldo);
```

```
  int get diaAniversario => _diaAniversario;

  // Sobrescrita de método (Polimorfismo)
  @override
  void exibirExtrato() {
    print("=== EXTRATO CONTA POUPANÇA ===");
    print("Titular: $titular");
    print("Dia do Rendimento: $_diaAniversario");
    print("Saldo atual: R\$ ${saldo.toStringAsFixed(2)}");
    print("================================");
  }

  // Método específico da Poupança
  void renderJuros(double taxa) {
    double juros = saldo * (taxa / 100);
    depositar(juros);
    print("Rendimento de R\$ ${juros.toStringAsFixed(2)} aplicado!");
  }
}

// ========================================
// 3. FUNÇÃO PRINCIPAL (EXECUÇÃO)
// ========================================
void main() {
  print("=== SISTEMA BANCÁRIO - ETEC MOBILE ===");

  // Criando um objeto do tipo ContaBancaria
  print("\n[Criando Conta Comum]");
```

```
 var contaComum = ContaBancaria("Carlos Silva", 500.0);

 contaComum.depositar(200.0);

 contaComum.sacar(100.0);

 contaComum.exibirExtrato();


 // Criando um objeto do tipo ContaPoupanca (Herança)

 print("\n[Criando Conta Poupança]");

 var contaPoup = ContaPoupanca("Ana Souza", 1200.0, 15);

 contaPoup.renderJuros(1.0); // Aplica 1% de juros

 contaPoup.exibirExtrato(); // Executa o método polimórfico sobrescrito

}
```

5. CONCLUSÃO DO RELATÓRIO

A implementação desenvolvida em **Dart** demonstra a aplicação de todos os conceitos da Orientação a Objetos:

1. **Abstração:** Garantida na classe ContaBancaria e nas interfaces Tributavel e Rentavel.
2. **Encapsulamento:** Visível nos atributos privados (prefixados com _) e acessados/modificados apenas por getters e métodos de validação.
3. **Herança:** Presente na extensão de ContaBancaria pelas classes ContaCorrente e ContaPoupanca.
4. **Polimorfismo:** Evidenciado na sobrescrita do método sacar() na ContaCorrente para incluir o limite do cheque especial e na implementação dos extratos específicos.
5. **Recursos Dart:** O código utilizou **Mixins** (Auditavel, ImpressoraComprovante), **Enums avançados** (TipoTransacao), **Tratamento de Exceções Customizadas** e **Sound Null Safety**.